🧩 Firmware & Protocol Specification

> Commands are in uppercase and mono-spaced, e.g. `VERIFY`, variables are bold e.g. **AlarmIndex** and other important text such as MQTT channels are *in Italics*.

Version	Date	Comments
1.0.3		
1.0.4	`Aug 24, 2023`	Added Water Valve `ALERT` type.

MQTT

- For all MQTT communications between the controller and server, **MQTT version 3.1.1** will be used.
- All commands will use **QoS 2 - Exactly Once** unless otherwise specified.
- All commands will **not be retained** unless otherwise specified.

Listen and Response topics

- The listen topic will be `sg/sas/cmd/{sn}` where `{sn}` is the serial number of the controller.
- The listen topic will be `sg/sas/resp/{sn}` where `{sn}` is the serial number of the controller.

Command format

All commands and their responses will be JSON formatted.

Summary of Commands

The following commands will be emitted from the controller:

- VERIFY - Verify that a controller has a connection.
- ADD - Link alarm to a controller.
- REMOVE - Unlink the alarm from a controller.
- BATTERY - Reporting battery life of alarms or a controller.
- ALARMS - Listing alarms linked to a controller.

- RESET - Reset a controller to factory settings, unlinking all linked alarms.
- REBOOT - Reboot a controller.
- POWER - Reporting power change events for a controller.
- DISCONNECT - Reporting an alarm disconnecting from a controller.
- RECONNECT - Reporting an alarm reconnecting to a controller.
- TEST - Testing alarms linked to a controller.
- LOW BATTERY - Reporting low battery warnings on alarms or a controller.
- TAMPERED - Reporting tamper switch changes on alarms.
- FAULT - Reporting alarm or controller faults.
- HUSH - Reporting hush switch changes on alarms.
- ALERT - Reporting alarms being triggered.

The following subset of commands will be emitted from the controller, triggered via an alarm.

- TEST
- BATTERY
- LOW BATTERY
- TAMPERED
- FAULT
- HUSH
- ALERT

Bridge Hardware

- Speaker to allow new functionality in the future.
- Red call box plastic w/ QR code on the front - see QR code section for details.
- Twilio Super SIM for connectivity.
- Wireless interconnect to alarm devices through the mesh network.
- Four LED indicators on the front.

LED Indicators

In order from left to right:

- AC Power - green, on and stable indicates AC power, and flashing indicates battery power.
- Fire - red.
- Fault - yellow, on and stable indicates alarm fault, flashing indicates hub fault.
- 4G (Wireless waves)

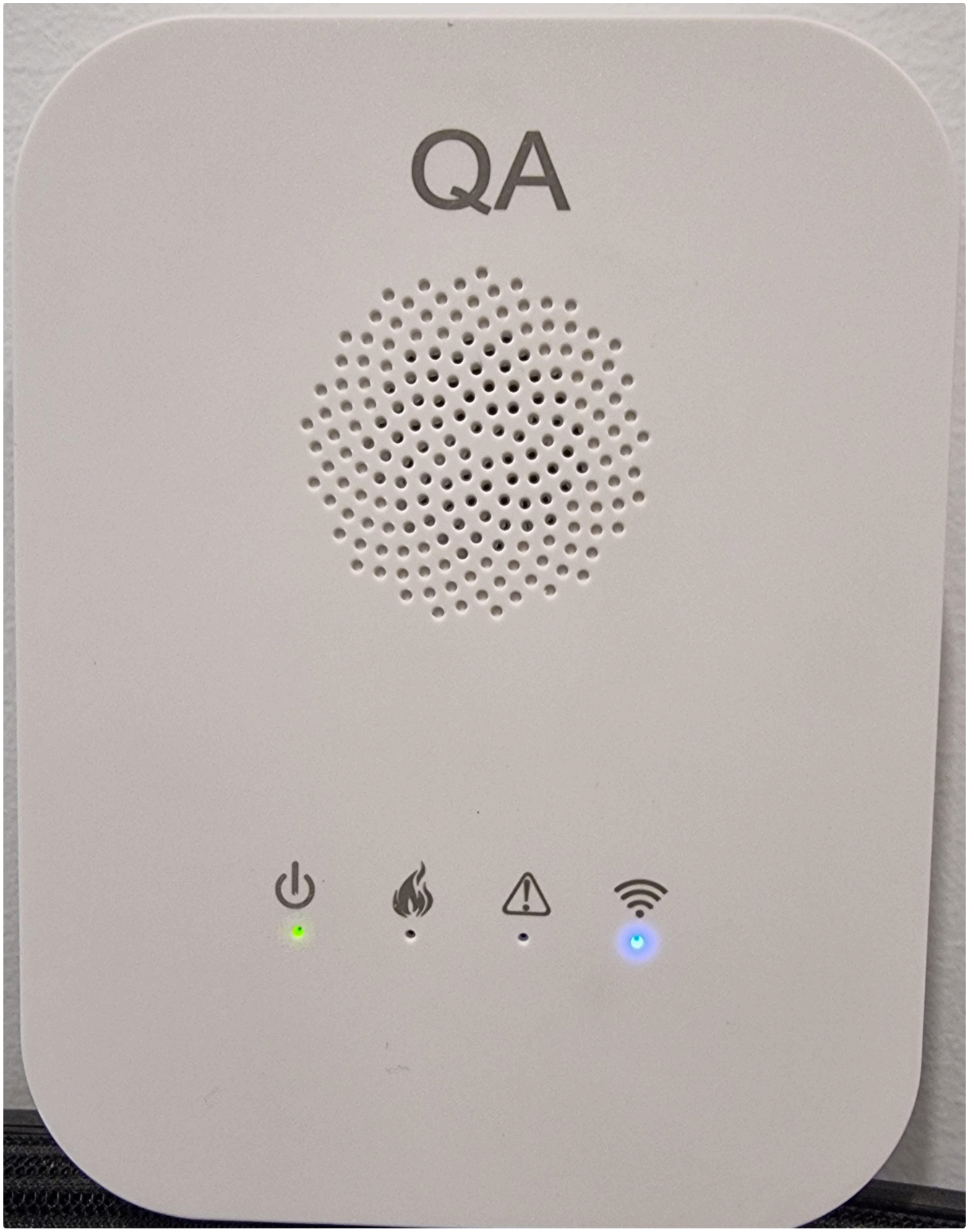
a. LED off: 4G module haven't ready, or can not be connected

b. flashing averagely: is connecting MQTT

c. 2 times flash quickly, and then stop one time: SIM card didn't detect

d. 3 times flash quickly, and then stop one time: is registering the internet

e. stable light on: indicates, already connected to the MQTT server well



Sensor Hub (controller)

Power

- AC power, with a backup battery.
- Aim for **six months** (In testing, this is around 48 hours) battery life on backup in case the controller loses power — this will be rechargeable when on AC power.

Communications

- Sub-GHz FSK 433MHz mesh node.
- Encryption for communications, so alarms cannot be triggered maliciously or accidentally via garage door openers, etc.

Buttons

Reset button

- A short press will REBOOT the controller.
- Long press for 10,000ms will RESET controller. See the RESET command for more details.

Lights

- One green LED - indicates AC power is connected.
- One red LED - indicates the ALARM state.

Ports

- An interface on the controller board, not externally accessible - to allow flashing firmware while in development.

Controller Commands

Heartbeat

The detector will periodically send a heartbeat message to the bridge every 5 mins. If a detector fails to send the heartbeat within 5 minutes, the controller will notify a `DISCONNECT` message.

VERIFY

The controller will handle an incoming `VERIFY` event on the *Command channel*:

```
{"CMD":"VERIFY"}
```

When received, the controller will emit a corresponding response `VERIFY` event on the *Response channel* with the controller's *Serial number*.

```
{"CMD":"VERIFY","CONTROLLERSERIAL":<ControllerSerial>,"STATUS":<BoolStatus>,"POWERSTATE":<PowerState>,"BATTERY":<BatteryPercentage>}
{"CMD":"VERIFY","CONTROLLERSERIAL":"97P86CGGC901C0012105","STATUS":1,"POWERSTATE":1,"BATTERY":100}
```

Where the **ControllerSerial** corresponds to the serial number of the controller, the **BatteryPercentage** will be an integer, corresponding to the controller's battery percentage, and **BoolStatus** is one of the following:

- `0` - Undefined
- `1` - Success
- `2` - Failure

And the **PowerState** is one of the following:

- `0` - Undefined
- `1` - AC Power
- `2` - Backup battery

ADD

The controller will handle an incoming `ADD` event on the *Command channel* to link an alarm to the controller:

```
{"CMD":"ADD","ALARMSERIAL":<AlarmSerial>}
```

When received, the controller will emit a corresponding response `ADD` event on the *Response channel* with the alarm's *Alarm index*.

```
{"CMD":"ADD","ALARMSERIAL":<AlarmSerial>,"INDEX":<AlarmIndex>,"STATUS":
<BoolStatus>,"BATTERY":<BatteryPercentage>}
{"CMD":"ADD","ALARMSERIAL":"AAABBBCCCABCA0012104","INDEX":0,"STATUS":1,"BATTERY":75}
```

Where the **AlarmSerial** corresponds to the serial number of the alarm, the **AlarmIndex** corresponds to the controller's index for that alarm, and **BoolStatus** is one of the following:

- `0` - Undefined
- `1` - Success
- `2` - Failure

If the **AlarmSerial** already exists, the response will be `2` to indicate failure, and no **AlarmIndex** will be emitted as follows:

```
{"CMD":"ADD","ALARMSERIAL":"AAABBBCCCABCA0012104","STATUS":2}
```

> **NOTE:** The delay when the controller searches for alarms will be longer unless we use button presses to put the alarm into pairing mode. The delay will be up to **ten seconds**.

REMOVE

The controller will handle an incoming `REMOVE` event on the *Command channel* to unlink an alarm from the controller. The alarm to remove can be specified by either `INDEX` or `ALARMSERIAL`, which will remove the alarm in the **AlarmIndex**-th position or the one with **AlarmSerial** as its serial number, respectively. If both `INDEX` and `ALARMSERIAL` are specified, `INDEX` will take precedence.

```
{"CMD":"REMOVE","ALARMSERIAL":<AlarmSerial>}
{"CMD":"REMOVE","INDEX":<AlarmIndex>}
```

When received, the controller will emit a corresponding response `REMOVE` event on the *Response channel* indicating success or failure, and both the `INDEX` and `ALARMSERIAL` of the alarm were removed.

```
{"CMD":"REMOVE","INDEX":<AlarmIndex>,"ALARMSERIAL":<AlarmSerial>,"STATUS":<BoolStatus>}
{"CMD":"REMOVE","INDEX":1,"ALARMSERIAL":"AAABBBCCCABCA0012104","STATUS":1}
```

Where the **AlarmSerial** corresponds to the serial number of the alarm, the **AlarmIndex** corresponds to the controller's index for that alarm, and **BoolStatus** is one of the following:

- `0` - Undefined
- `1` - Success
- `2` - Failure

If both the **AlarmIndex** or **AlarmSerial** does not correspond to an alarm linked to the controller, the response will be `2` to indicate failure, and the `INDEX` or `ALARMSERIAL` will be included as per the request as follows:

```
{"CMD":"REMOVE","INDEX":1,"STATUS":2}
{"CMD":"REMOVE","ALARMSERIAL":"AAABBBCCCABCA0012104","STATUS":2}
```

TEST

The controller will handle an incoming `TEST` event on the *Command channel* that loops through all alarms linked to the controller and test each of them individually in sequence.

A test will involve sounding the alarm and using the *microphone* to measure the dB and reporting success for correct values. Failure if the *microphone* does not register the alarm sounding or the controller does not get a response from the alarm.

An optional `INDEX` can be specified, which will limit the test to that one alarm in the **AlarmIndex**-th position:

```
{"CMD":"TEST"}
{"CMD":"TEST","INDEX":<AlarmIndex>}
```

An example where only the *alarm with index 5* will be tested is `{"CMD": "TEST", "INDEX":5}`.

When received, the controller will respond with *zero or more* corresponding `TEST` events on the *Response channel*, one per alarm known to the controller:

```
{"CMD":"TEST","INDEX":<AlarmIndex>,"STATUS":<BoolStatus>}
```

Where the **AlarmIndex** will be an integer corresponding to the index of the alarm and the **BoolStatus** is one of the following:

- `0` - Undefined
- `1` - Success
- `2` - Failure.

Some examples of the format are as follows:

```
{"CMD":"TEST","INDEX":0,"STATUS":1}
{"CMD":"TEST","INDEX":1,"STATUS":1}
{"CMD":"TEST","INDEX":3,"STATUS":1}
{"CMD":"TEST","INDEX":2,"STATUS":2}
```

For an additional overview, there are two ways of testing alarms: the physical button or via MQTT:

1. Physical *Test Button* pressed on Alarm.
2. The alarm runs a self-test and reports back to Controller.
3. The controller sends `{"CMD": "TEST", "INDEX":<AlarmIndex>, "STATUS": <BoolStatus>}` response to the backend server.

Or:

1. The controller receives `{"CMD": "TEST"}` or `{"CMD": "TEST", "INDEX": <AlarmIndex>}` command.
2. The controller causes all Alarms to test (the first command with no index specified) or one individual Alarm to test (the second command with an index specified).
3. For each Alarm that performs a test, the Controller will send back a `{"CMD": "TEST", "INDEX":<AlarmIndex>, "STATUS":<BoolStatus>}` response to the backend server.

BATTERY

The controller will handle an incoming `BATTERY` event on the *Command channel* that will emit the current battery percentage of all alarms currently known to the controller.

An optional `INDEX` can be specified, which will limit the battery check to that one alarm in the **AlarmIndex**-th position:

```
{"CMD":"BATTERY"}
{"CMD":"BATTERY","INDEX":<AlarmIndex>}
```

An example where only the *alarm with index 5* will be checked is `{"CMD": "BATTERY", "INDEX":5}`.

When received, the controller will respond with *zero or more* corresponding `BATTERY` events on the *Response channel*, one per alarm known to the controller:

```
{"CMD":"BATTERY","INDEX":<AlarmIndex>,"BATTERY":<BatteryPercentage>}
```

Where the **AlarmIndex** will be an integer corresponding to the index of the alarm and the **BatteryPercentage** will also be an integer, corresponding to the alarm's battery percentage. Some examples of the format are as follows:

```
{"CMD":"BATTERY","INDEX":0,"BATTERY":15}
{"CMD":"BATTERY","INDEX":1,"BATTERY":99}
{"CMD":"BATTERY","INDEX":3,"BATTERY":100}
{"CMD":"BATTERY","INDEX":2,"BATTERY":73}
```

ALARMS

The controller will handle an incoming `ALARMS` event on the *Command channel* that will list all detectors currently known to the controller:

```
{"CMD":"ALARMS"}
```

When received, the controller will respond with a corresponding `ALARMS` event on the *Response channel*:

```
{"CMD":"ALARMS","ALARMLIST":<ListOfAlarms>}
```

Where the **ListOfAlarms** will be a string containing a *comma-delimited* list of detector keys and values *separated by semi-colons*. Some examples of the format are as follows:

```
{"CMD":"ALARMS","ALARMLIST":"ALARMKEY;ALARMSERIAL,ALARMKEY;ALARMSERIAL"}
{"CMD":"ALARMS","ALARMLIST":"0;AAAA,1;BBBB,2;CCCC"}
{"CMD":"ALARMS","ALARMLIST":"0;97P86CGGC901A0012104,1;J7P46PP9C901A0012104,2;AB126C99C900A001
2104,3;ALARMSERIALXA0012104"}
```

RESET

The controller will handle an incoming `RESET` event on the *Command channel*:

```
{"CMD":"RESET"}
```

When received, the controller will emit a corresponding response `RESET` event on the *Response channel* and promptly reset to factory settings and reboot. This will unlink all linked alarms.

```
{"CMD":"RESET","STATUS":<BoolStatus>}
```

Where the **BoolStatus** is one of the following:

- `0` - Undefined
- `1` - Success
- `2` - Failure

REBOOT

The controller will handle an incoming `REBOOT` event on the *Command channel*:

```
{"CMD":"REBOOT"}
```

When received, the controller will emit a corresponding response `REBOOT` event on the *Response channel* and promptly reboot.

```
{"CMD":"REBOOT","STATUS":<BoolStatus>}
```

Where the **BoolStatus** is one of the following:

- `0` - Undefined
- `1` - Success
- `2` - Failure

POWER

As the controller changes power state, a `POWER` event will be emitted by the controller on the *Response channel*:

```
{"CMD":"POWER","POWERSTATE":<PowerState>}
```

Where the **PowerState** is one of the following:

- `0` - Undefined
- `1` - AC Power
- `2` - Backup battery

When the controller transitions from *AC power* to the *Backup battery*, `{"CMD": "POWER", "POWERSTATE":2}` and when transitioning back from *Backup battery* to *AC power*, `{"CMD": "POWER", "POWERSTATE":1}`.

LOW_BATTERY

If the controller or any linked alarm passes downwards through specified thresholds, a `LOW_BATTERY` event will be emitted by the controller on the *Response channel*. Any alarm-originating low battery events will include the optional `INDEX` value corresponding to the *Alarm index*. All events will include the **BatteryPercentage** percentage.

The thresholds for low battery events will be 5%, 10% and 15%.

```
{"CMD":"LOW_BATTERY","BATTERY":<BatteryPercentage>}
{"CMD":"LOW_BATTERY","INDEX":<AlarmIndex>,"BATTERY":<BatteryPercentage>}
```

Where the **AlarmIndex** will be an integer corresponding to the index of the alarm and the **BatteryPercentage** will also be an integer, corresponding to the remaining percentage of the battery. Some examples of the format are as follows:

```
{"CMD":"LOW_BATTERY","BATTERY":15}
{"CMD":"LOW_BATTERY","INDEX":3,"BATTERY":30}
```

TAMPERED

If any linked alarm changes the state of its tamper switch, a `TAMPERED` event will be emitted by the controller on the *Response channel*, including the `INDEX` value corresponding to the *Alarm index*:

```
{"CMD":"TAMPERED","INDEX":<AlarmIndex>,"STATE":<BoolState>}
```

Where the **AlarmIndex** corresponds to the index of the linked alarm, and **BoolState** is one of the following:

- `0` - Undefined
- `1` - On
- `2` - Off

When the alarm with index 2's tamper switch transitions from *On* to *Off*, `{"CMD": "TAMPERED", "INDEX":2, "STATE":2}` and when transitioning back from *Off* to *ON*, `{"CMD": "TAMPERED", "INDEX":2, "STATE":1}`.

RECONNECT

If any linked alarm reconnects to the controller network via a successful heartbeat following any unsuccessful heartbeat, a `RECONNECT` event will be emitted by the controller on the *Response channel*, including the `INDEX` value corresponding to the *Alarm index*:

```
{"CMD":"RECONNECT","INDEX":<AlarmIndex>}
```

Where the **AlarmIndex** corresponds to the index of the linked alarm, e.g. the alarm with index 3 comes back online, `{"CMD": "RECONNECT", "INDEX":3}` will be emitted.

DISCONNECT

If any linked alarm disconnects from the controller network via an unsuccessful heartbeat, a `DISCONNECT` event will be emitted by the controller on the *Response channel*, including the `INDEX` value corresponding to the *Alarm index*:

```
{"CMD":"DISCONNECT","INDEX":<AlarmIndex>}
```

Where the **AlarmIndex** corresponds to the index of the linked alarm, e.g. the alarm with index 3 does not return a heartbeat, `{"CMD": "DISCONNECT", "INDEX":3}` will be emitted.

FAULT

If the controller or any linked alarm self-diagnoses a fault, a `FAULT` event will be emitted by the controller on the *Response channel*. Any alarm-originating faults will include the optional `INDEX` value corresponding to the *Alarm index*.

```
{"CMD":"FAULT","CODE":<FaultCode>}
{"CMD":"FAULT","INDEX":<AlarmIndex>,"CODE":<FaultCode>}
```

Where the **AlarmIndex** will be an integer corresponding to the index of the alarm and the **FaultCode** will also be an integer, corresponding to a known list of fault codes. Some examples of the format are as follows:

```
{"CMD":"FAULT","CODE":5}
{"CMD":"FAULT","INDEX":3,"CODE":3}
```

Fault codes will be one of the following:

- **10**: Cleaning required.

HUSH

If any linked alarm changes the state of its hush button, a `HUSH` event will be emitted by the controller on the *Response channel*, including the `INDEX` value corresponding to the *Alarm index*:

```
{"CMD":"HUSH","INDEX":<AlarmIndex>,"STATE":<BoolState>}
```

Where the **AlarmIndex** corresponds to the index of the linked alarm, and **BoolState** is one of the following:

- `0` - Undefined
- `1` - On
- `2` - Off

When the alarm with index 2's hush button transitions from *On* to *Off*, `{"CMD": "HUSH", "INDEX":2, "STATE":2}` and when transitioning back from *Off* to *ON*, `{"CMD": "HUSH", "INDEX":2, "STATE":1}`.

ALERT

If any linked alarm changes its alarm state, an `ALERT` event will be emitted by the controller on the *Response channel*, including the `INDEX` value corresponding to the *Alarm index* and the

`ALARMTYPE` corresponding to the type of alarm event:

```
{"CMD":"ALERT","INDEX":<AlarmIndex>,"ALARMTYPE":<AlarmType>,"STATE":<BoolState>}
```

Where the **AlarmIndex** corresponds to the index of the linked alarm, the **AlarmType** corresponds to one of the following:

- `0` - Undefined
- `1` - Smoke
- `2` - Carbon Monoxide
- `3` - Water Leak
- `4` - Gas Leak
- `5` - Vibration
- `6` - Door
- `7` - Carbon Dioxide
- `8` - Refrigerant Leak
- `9` - Water Valve
- `10` - `49` - Reserved for additional sensors.

and **BoolState** is one of the following:

- `0` - Undefined
- `1` - On
- `2` - Off

For example, if the alarm with index 2 sounds due to smoke, `{"CMD": "ALERT", "INDEX":2, "ALARMTYPE":1, "STATE":1}` will be emitted. When this alarm is no longer sounding, `{"CMD": "ALERT", "INDEX":2, "ALARMTYPE":1, "STATE":2}` will be emitted.

217E-W01 Water Leak Sensor

- When a water leak sensor triggers an `ALERT`, they will also broadcast to all Water Shutoff Valves to activate.
- This broadcast will be in a 10 seconds on, 5 seconds off repeating pattern.

217E-02 Photoelectric Smoke Alarm

Photoelectric smoke alarm, attached controllers and other alarms via a wireless mesh network.

Alarm Hardware

- Photoelectric smoke alarm.
- 10-year non-replaceable battery and/or AC powered via both 120/240V.
- One 3V battery for wireless communication, and one 3V battery for alarm operation.
- The AC power will power the wireless communication when available.
- Fine mesh fabric and metal cage around photo-sensor to prevent false alarms by bug ingress.
- Waxy covering on PCBs to increase the lifespan of devices.
- Alarms will vary their sensitivity to minimise nuisance tripping.
- Alarms will throw a `FAULT` code **10** to indicate cleaning is required once the sensing chamber has been obstructed and is no longer able to adequately sense smoke.
- QR code on the front - see QR code section for details.

Microphone

- Each Alarm will have a microphone embedded in it, to enable local dB testing.

Buttons

Test button

- A long press will activate a test as with other fire alarms.
- After 1000ms start low volume alarm tone for between 1000ms and 3000ms, then maximum volume tone from 3000ms.
- After a test, the alarm will notify the Controller in the same manner as a Controller-prompted test. See the Controller `TEST` command for more details.

Hush button

- Pressing will hush the alarm.

Ports

- An interface on the alarm board, not externally accessible - to allow flashing firmware while in development.

Alarm Commands

Alarm commands are a subset of controller commands.

Heartbeat

Each alarm will immediately respond to a controller's heartbeat message with an acknowledgement response.

TEST (Alarm)

When prompted by the controller, an alarm will initiate a self-test. See the Controller `TEST` command for more details.

BATTERY (Alarm)

Alarms will respond with their battery life when queried by the controller. See the Controller `BATTERY` command for more details.

LOW_BATTERY (Alarm)

Alarms will notify the controller when passing below low battery thresholds. See Controller `LOW_BATTERY` command for more details.

TAMPERED (Alarm)

Alarms will notify the controller when their tamper switch changes state. See the Controller `TAMPERED` command for more details.

While in the tamper state, the alarm will *chirp*.

FAULT (Alarm)

Alarms will notify the controller if any faults occur. See the Controller `FAULT` command for more details.

HUSH (Alarm)

Alarms will notify the controller when their hush button changes state. See the Controller `HUSH` command for more details.

ALERT (Alarm)

Alarms will notify the controller when their alarm state changes. See the Controller `ALERT` command for more details.

Alarm Indices

When linked, alarms will be referenced through integer keys. When unlinking alarms, their index will be re-used when new alarms are added.

For instance, if three alarms are added, with the serials of `AAA`, `BBB` and `CCC` to a new controller, the controller will contain the following:

- `0` - AAA
- `1` - BBB
- `2` - CCC

If the alarm with index `1` is unlinked, the controller will then contain the following:

- `0` - AAA
- `2` - CCC

Upon adding two additional alarms with the serials of `DDD` and `EEE`, the controller will then contain the following:

- `0` - AAA
- `1` - DDD
- `2` - CCC
- `3` - EEE

QR Codes

The QR code on all devices will be a URL that corresponds to the activation URL combined with the 20-character serial number:

- https://activation.sensorglobal.com/?s=97P86CGGC901C0012105
- https://activation.sensorglobal.com/?s=AAABBBCCCABCA0012104
- https://activation.sensorglobal.com/?s=AAAABBBBCCCCA0022103
- https://activation.sensorglobal.com/?s=AAAABBBBCC01A0032101
- https://activation.sensorglobal.com/?s=AAAABBBBCC02A0042101

Under the QR code, the serial number will be displayed.

Serial numbers

Serial numbers will be a 20-character field as follows, with hyphenation added in some examples for readability.

- 12-character unique serial number.
- 4-character make/model code, see below for details.
- 4-character manufacture date in *YYMM* format.

i.e. XXXXXXXXXXXX-XXXX-XXXX or 97P86CGGC901-C001-2105 (With hyphens for readability.)
i.e. XXXXXXXXXXXXXXXXXXXX or 97P86CGGC901C0012105

Dates will be used for expiry calculations at month-granularity, so all expiry calculations will only be accurate to the month. Two-digit years will mean we can only produce these up until the year 2121 before we need to revisit this.

Make/Model codes

The model will correspond to the following table with room for 26000 products before this needs to be revisited.

- C001 - 217E-C001 Sensor Hub (AU)
- A001 - 217E-02 Photoelectric Smoke Alarm (AU)
- A002 - Carbon Monoxide Alarm
- A003 - 217E-03 Photoelectric Smoke Alarm and Carbon Monoxide Alarm (US)
- A004 - 217E-W01 Water Leak Sensor
- A005 - 217E-W02 Water Shutoff Valve
- ...
- A006 - A999 - Additional models.